Санкт-Петербургский государственный университет

Кафедра, на которой работает научник

Группа XX.БXX-мм

Шаблон отчёта по учебной практике

ФАМИЛИЯ Имя Отчество

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н. (если есть), Научник Н. Н.

Консультант:
должность, ООО «Место работы», степень (если есть), Консультант К. К.

Санкт-Петербург
2026

Оглавление

Введение **4**

1. Постановка задачи **5**

2. Обзор **6**

2.1. Обзор существующих решений 6

2.2. Обзор используемых технологий 7

2.3. Выводы 8

3. Описание решения **9**

3.1. Первая задача 10

3.2. Вторая задача 10

3.3. Третья задача 10

3.4. Некоторые типичные ошибки 10

3.5. Листинги, картинка и прочий «не текст» 12

3.6. Некоторые детали описания реализации 13

4. Эксперимент **17**

4.1. Условия эксперимента 17

4.2. Исследовательские вопросы 17

4.3. Метрики 18

4.4. Результаты 18

4.5. Обсуждение результатов 20

4.6. Угрозы нарушения корректности (опциональный) 20

4.7. Воспроизводимость эксперимента 20

Заключение **22**

Список литературы **23**

Текст ВКР или учебной практики пишется не ради зачета, а чтобы люди его прочитали, поняли как круто Вы все сделали, и могли продолжить с того места, где Вы остановились.

Повторять эту страницу в тексте вашей работы *нельзя*.

Введение

Формат из 4х частей рекомендуется в курсе Д. Кознова [5] по написанию текстов.

1. Известная информация (background/обзор).
2. Неизвестная информация (пробел в знаниях, «Gap»).
3. Гипотезы, вопросы, цели — «что болит», что будет решать Ваша работа.
4. Подход, план решения задачи, предлагаемое решение.

Последний абзац должен читаться и быть понятен в отрыве от других трёх. Никакие абзацы нумеровать нельзя.

Части (абзацы) должны занять максимум две страницы, идеально уложиться в одну.

С.-П. Джонс [4] предлагает несколько другой формат написания введения. Вполне возможно, что если Ваша работа про языки программирования, то его формат будет удачнее.

Введение и заключение читают чаще всего, поэтому они должны быть «вылизаны» до блеска.

Иногда рецензенту полезно знать какого числа компилировался текст, чтобы оценить актуальность версии текста. В этом случае полезно вставлять в текст дату сборки. Для совсем официальных релизов документа это не вполне канон.
Также здесь имеет смысл указать, если работа сделана на деньги, например, Российского Фонда Фундаментальных Исследований (РФФИ) по гранту номер такой-то, и т.п.

1. Постановка задачи

Дословно «Целью работы является... Для её выполнения были поставлены следующие задачи:»

1. реализовать это (раздел 3.1);
2. спроектировать то-то (раздел 3.2) наилучшим образом;
3. протестировать на том-то (раздел 3.3) и обогнать тех-то;
4. ~~изучить язык ОСаml~~ писать тут не надо, так как тут должны быть задачи, выполнение которых можно проверить/оценить прочитав текст или выслушав доклад; (т.е. Ваши достижения должны быть опровержимы)
    - это может вызвать сомнения по поводу обзора — *выполнить обзор* писать можно и нужно, но защищаемым результатом будут не ваши знания, а текст обзора (то есть он должен иметь ценность сам по себе);
5. обязательна задача на валидацию результата, будь то эксперимент, апробация, внедрение — то есть доказательство того, что Вы сделали что-то, нужное пользователю. Не путайте с валидацией — доказательством того, что Вы сделали то, что хотели Вы (например, тесты — валидация результата, хорошо, но недостаточно).

2. Обзор

В данном разделе нужно описать всё, что необходимо для понимания Вашей работы и что придумали не Вы. В дальнейших разделах нельзя прерывать повествование, например, для рассказа о деталях используемой технологии или архитектуре старой системы, потому что читателю будет трудно отличить Ваш вклад от не Вашего.

Любой обзор пишется с какой-то целью (обосновать актуальность, найти и описать интересные решения, сравнить и выбрать технологии) и по какой-то методике поиска материала (например, поиск N релевантных статей на таких-то сервисах). Не будет лишним это всё явно описать.

2.1. Обзор существующих решений

Обзор существующих решений должен быть. Здесь нужно писать, что индустрия и наука уже сделали по вашей теме. Он нужен, чтобы Вы случайно не изобрели какой-нибудь велосипед.

По-английски называется related works или previous works.

Если Ваша работа является развитием предыдущей и плохо понимаема без неё, то обзор должен идти в начале. Если же Вы решаете некоторую задачу новым интересным способом, то если поставить обзор в начале, то читатель может устать, пока доберется до вашего решения. В этом случае уместней поставить обзор после описания Вашего подхода к проблеме[1].

В обзоре вам нужно рассказать про *преимущества и недостатки* того, что было сделано до Вас. Неправильным будет перечислять только недостатки, так как если Ваша работа хоть где-то хуже предыдущей, то рецензент будет радостно потирать руки и заваливать Вашу работу. Гораздо лучше, если Вы честно признаетесь в этом сами.

[1]Такой подход рекомендуется в работе [4]. Вполне возможно, что Ваш реальный научный руководитель будет не согласен, и потребует, чтобы обзор был в начале.

2.2. Обзор используемых технологий

Для технических работ обзор может обозревать продукт, в рамках которого Вы выполняете задачу, другие продукты, где решалась схожая задача, а также используемые технологии с обоснованием выбора тех, которые Вы дальше используете. Это всё, скорее всего, будет отдельными подразделами обзора. «Выбор» подразумевает наличие вариантов, поэтому опишите, из чего выбирали и почему выбрали то, что выбрали. Очень желательны чёткие критерии сравнения и сводная таблица в конце, где стоят плюсы и минусы рядом с каждым рассматриваемым вариантом.

В обзоре необходимо ссылаться на работы других людей. В данном шаблоне задумано, что литература будет указываться в файле `vkr.bib`. В нём указываются пункты литературы в формате BibTeX, а затем на них можно ссылаться с помощью `\cite{...}`. Та литература, на которую Вы сошлетесь, попадет в список литературы в конце документа. Если не сошлетесь — не попадёт. Спецификацию в формате BibTeX почти никогда (для второго курса — никогда), не нужно придумывать руками. Правильно: находить в интернете описание цитируемой статьи[2], копировать цитату с помощью кнопки “Export Citation” и вставлять в BibTeX файл. Так же умеет генерировать BibTeX-описания и Google Scholar[3]. Если так не делать, то оформление литературы будет обрастать ошибками. Например, BibTeX по особенному обрабатывает точки, запятые и `and` в списке авторов, что позволяет ему самому понимать, сколько авторов у статьи, и что там фамилия, что — имя, а что — отчество. Google Scholar пытается генерировать описания автоматически, так что, возможно, потребуется ручная правка — обязательно проверьте свой список литературы.

В обзоре и в остальном тексте вы наверняка будете использовать названия продуктов или языков программирования (например, C#). Для них рекомендуется (в файле `preamble2.tex`) задать специаль-

[2]Например, https://dl.acm.org/doi/10.1145/3408995 (дата обращения: 17 декабря 2022 г.).

[3]Поисковая система для научных текстов Google Scholar, https://scholar.google.com (дата обращения: 13 января 2024 г.)

ные команды, чтобы писать сложные названия правильно и одинаково по всему документу. Написать с ошибкой название любимого языка программирования научного руководителя — идеальный вариант его разозлить.

2.3. Выводы

Опишите явно, что читатель должен был вынести из обзора в отдельном подразделе.

3. Описание решения

Реализация в широком смысле: что таки было сделано. Часто это на самом деле несколько отдельных разделов, по одному на каждую «реализационную» задачу из постановки. Например, «Архитектура» и «Особенности реализации», либо по разделу на каждую крупную подзадачу. Если разделы получились недостаточно эпичными (меньше пары страниц), сделайте их подразделами одного большого раздела, как тут.

Если программной реализации нет, тут пишутся Ваши теоретические наработки, если есть, хоть в каком-то виде, то опишите, даже если серьёзно кодом надо будет заняться только в следующей части.

Если работа на текущем этапе предполагает *только* обзор, то

- но всё равно же надо было попробовать воспроизвести чужие результаты, напишите про это сюда;
- если нет, то, наверное, обзор получился годным и ценным сам по себе, источников 40–50 было рассмотрено[4], тогда ладно, раздел «Описание решения» можно не делать.

Для понимания того, как отчёт по учебной практике должен писаться, можно посмотреть видео ниже. Они про научные доклады и написание научных статей. Учебные практики и ВКР отличаются тем, что тут есть требования отдельных глав (слайдов) со списком задач и списком результатов. Но даже если Вы забьёте на требования, специфичные для ВКР, и соблюдете все рекомендации ниже, получившиеся скорее всего будет лучше, чем первая попытка 99% ваших однокурсников.

1. «Как сделать великолепный научный доклад» от Саймона Пейтона Джонса [3] (на английском).
2. «Как написать великолепную научную статью» от Саймона Пейтона Джонса [4] (на английском).

[4]Это не шутка, в хороших работах, где целый семестр делался только обзор, оно примерно так и выходит.

3. «Как писать статьи так, чтобы люди их смогли прочитать» от Дэрэка Драйера [1] (на английском). По предыдущим ссылкам разбирается, что должно быть в статье, т.е. как она должна выглядеть на высоком уровне. Тут более низкоуровнево изложено, как должны быть устроены параграфы и т.п.

4. Ещё можно посмотреть How to Design Talks [2] от Ranjit Jhala, но мы думаем, что первых трёх ссылок всем хватит.

Можно глянуть примеры хороших работ прошлых лет на сайте кафедры системного программирования[5]. Например, работа Москаленко Ивана Николаевича за 3-й курс[6].

3.1. Первая задача

3.2. Вторая задача

3.3. Третья задача

3.4. Некоторые типичные ошибки

Здесь мы будем собирать основные ошибки, которые случаются при написании текстов. В интернетах тоже можно найти коллекции типичных косяков[7].

Рекомендуется по-умолчанию использовать красивые греческие буквы σ и φ, а именно φ вместо ϕ. В данном шаблоне команды для этих букв переставлены местами по сравнению с ванильным TEX'ом.

Также, если работа пишется на русском языке, необходимо, чтобы работа была написана на *грамотном* русском языке даже если автор из ближнего зарубежья[8]. Это включает в себя:

[5]Сайт кафедры СП, архив практик и ВКР: URL: https://se.math.spbu.ru/theses.html (дата обращения: 15 января 2024 г.).

[6]«Создание базы данных изображений для глобальной локализации в пространстве», URL: https://se.math.spbu.ru/thesis_download?thesis_id=1199 (дата обращения: 15 января 2024 г.).

[7]https://www.read.seas.harvard.edu/~kohler/latex.html (дата обращения: 16 декабря 2022 г.).

[8]Теоретически, возможен вариант написания текстов на английском языке, но это необходимо обсудить в первую очередь с научным руководителем.

- разделы должны оформляться с помощью `\section{...}`, а также `\subsection` и т. п.; не нужно пытаться нумеровать вручную;
- точки после окончания предложений должны присутствовать;
- пробелы после запятых и точек, в конце слова перед скобкой;
- неразрывные пробелы там, где нужны пробелы, но переносить на другую строку нельзя, например `т.~е.`, `А.~Н.~Терехов`, `что-то~\cite{?}`, `что-то~\ref{?}`;
- дефис там, где нужен дефис (обозначается с помощью одиночного «минуса» (англ. dash) на клавиатуре);
- двойной дефис там, где он нужен; а именно при указании промежутка в цифрах: в 1900–1910 г. г., стр. 150–154;
- тире (т. е. `---` — тройной минус) на месте тире, а не что-то другое; в русском языке тире не может «съезжать» на новую строку, поэтому стоит использовать такой синтаксис: `До~--- после`;
- даты стоит писать везде одинаково; чтобы об этом не следить, можно пользоваться заклинанием `\DTMdate{2022-12-16}`;
- правильные кавычки должны набираться с помощью пакета `csquotes`: для основного языка в `polyglossia` стоит использовать команду `\enquote{текст}`, для второго языка стоит использовать `\foreignquote{язык}{текст}`; правильные кавычки в русской типографии — `<<ёлочки>>`, ни в коем случае не `"скандинавские лапки"`;
- все перечисления должны оформляться с помощью `\enumerate` или `\itemize`; пункты перечислений должны либо начинаться с заглавной буквы и заканчиваться точкой, либо начинаться со строчной и закачиваться точкой с запятой; последний пункт перечислений всегда заканчивается точкой.

- Перед выкладыванием финальной версии необходимо просмотреть лог LaTeX'а, и обратить внимание на сообщения вида *Overfull \hbox (1.29312pt too wide) in paragraph*. Обычно, это означает, что текст выползает за поля, и надо подсказать, как правильно разделять слова на слоги, чтобы перенос произошел автоматически. Это делается, например, так: `соломо\-волокуша`.

- *Обязательно используйте инструменты автоматической проверки правописания!* Не посылайте текст даже научному руководителю на проверку, если не прогнали спеллчекер[9], желательно, умеющий проверять пунктуацию — у научника куча времени уйдёт на комментарии вида «тут нужна запятая» и не останется сил посоветовать что-нибудь по делу. А ещё научник будет шокирован уровнем грамотности современной молодёжи, впадёт в депрессию и не будет отвечать вам неделю.

3.5. Листинги, картинка и прочий «не текст»

Различный «не текст» имеет свойство отображаться не там, где он написан в текстовом виде в LaTeX, поэтому у него должна быть самодостаточная понятная подпись `\caption{...}`, уникальная метка `\label{...}`, чтобы на неё можно было бы ссылаться в тексте с помощью `\ref{...}` (более того, ссылка из текста обязательна). Ниже Вы сможете увидеть таблицу 1, на которую мы сослались буквально только что.

«Не текста» может быть довольно много — чтобы не засорять корневую папку, хорошим решением будет складывать весь «не текст» в папку `figures`. Заклинание `\includegraphics{}` уже знает этот путь и будет искать там файлы без указания папки. Команда `\input{}` умеет ходит по путям, например `\input{figures/my_awesome_table.tex}`. Кроме того, листинги кода можно подтягивать из файла с помощью команды `\inputminted{file}`.

[9]Например, для браузера можно использовать плагин LanguageTool, для VS Code — Code Spell Checker с расширением для русского (но он вроде не умеет пунктуацию, так что следите за запятыми).

Листинг 1: Название для листинга кода. Достаточно длинное, чтобы люди, которые смотрят картинку сразу после названия статьи (т. е. все люди), смогли разобраться и понять к чему в статье листинги, картинки и прочий «не текст».

```
    let x = 5 in x + 1
```

3.5.1. Выделение куска листинга с помощью tikz

Это бывает полезно в текстах, а ещё чаще — в презентациях. Пример сделан на основе вопроса на StackExchange[10]. Заодно тут показывается альтернативный minted пакет, lstlistings, если не хотите ставить Python и пакет pygments. В этом случае закомментируйте всё, что связано с minted, в matmex-diploma-custom.cls. И внимательно следите за тем, чтобы везде использовался либо только lstlistings, либо minted — смешивание их в одном документе приведёт к странным ошибкам.

3.6. Некоторые детали описания реализации

Описание реализации — очень важный раздел для будущих программных инженеров, т.е. почти для всех. Важно иметь всегда, даже если Вы написали прототип на коленке или немного скриптов.

В процессе работы можно сделать огромное количество косяков, неполный список которых ниже.

1. Реализация должна быть. На публично доступную реализацию обязательна ссылка (в заключении, но можно продублировать тут). Если код под NDA, то об этом, во-первых, должно быть сказано явно, во-вторых, на защиту должны выноситься другие результаты (например, архитектура), чтобы комиссия имела возможность оценить хоть что-то, и, в третьих, должна быть справка от работодателя, что Вы правда что-то сделали.

[10]Вопрос про рамку вокруг листинга на StackExchange, https://tex.stackexchange.com/questions/284311 (дата обращения: 16 декабря 2022 г.).

```c
#include <stdio.h>
#include <math.h>

/** A comment in English */
int main(void)
{
  double c = -1;
  double z = 0;

  // Это комментарий на русском языке
  printf ("For␣c␣=␣%lf:\n", c);
  for (int i = 0; i < 10; i++ ) {
    printf( "z␣%d␣=␣%lf\n", i, z);
    z = pow(z, 2) + c;
  }
}
```

Рис. 1: Пример листинга с помощью пакета `listings` и TIKZ декорации к нему, оформленные в окружении `figure`. Обратите внимание, что рисунок отображается не там, где он в документе, а может «плавать».

- Рецензент обязан оценить код (о возможности должен побеспокоиться обучающийся).

2. Код реализации должен быть написан защищающимся целиком.

   - Если проект групповой, то нужно явно выделить, какие части были модифицированы защищающимся. Например, в предыдущих разделах на картинке архитектуры нужно выделить цветом то, что Вы модифицировали.
   - Нельзя пускать в негрупповой проект коммиты от других людей, или людей не похожих на Вас. Например, в 2022 году защищающийся-парень делал коммиты от сценического псевдонима, который намекает на женский «гендер». (Нет, это не шутка.) На тот момент в российской культуре это выглядело странно.
   - Возможна ситуация, что вы используете конкретный ник в интернете уже лет пять, и желаете писать ВКР под этим ником на GitHub. В принципе, это допустимо, но если Вы встретите преподавателя, который считает наоборот, то Вам придется грамотно отмазываться. В Вашу пользу могут сыграть те факты, что к нику на GitHub у Вас приписаны настоящие имя и фамилия; что в репозитории у вас видна домашка за первый курс; и что Ваш преподаватель практики сможет подтвердить, что Вы уже несколько лет используете это ник; и т.п.

3. Если Вы получаете диплом о присвоении квалификации программиста, код должен соответствовать.

   (a) Не стоит выкладывать код одним коммитом.

   (b) Не стоит выкладывать код аккурат перед защитой.

   (c) Лучше хоть какие-то тесты, чем совсем без них. В идеале нужно предъявлять процент покрытия кода тестами.

(d) Лучше сделать CI, а также CD, если оно уместно в Вашем проекте.

(e) Не стоит демонстрировать на защите, что Вам даже не пришло в голову напустить на код линтеры и т.п.

4. Если ваша реализация по сути является прохождением стандартного туториала, например, по отделению картинок кружек от котиков с помощью машинного обучения, то необходимо срочно сообщить об этом руководителю практики/ВКР, иначе Государственная Экзаменационная Комиссия «порвёт Вас как Тузик грелку», поставит «единицу», а все остальные Ваши сокурсники получат оценку выше. (Это не шутка, а реальная история 2020 года.)

Если Вам предстоит защищать учебную практику, а эти рекомендации видятся как более подходящие для защиты ВКР, то ... отмаза не засчитывается, сразу учитесь делать нормально.

4. Эксперимент

Как мы проверяем, что всё удачно получилось. Если работа рассчитана на несколько семестров и в текущем до эксперимента дело не дошло, опишите максимально подробно, как он будет проводиться и на чём (то, что называется *дизайн эксперимента* — от того, что и как Вы будете проверять, очень сильно зависит, что Вы будете делать, так что это важно и делается *не* после реализации).

4.1. Условия эксперимента

Железо (если актуально); версии ОС, компиляторов и параметры командной строки; почему мы выбрали именно эти тесты; входные данные, на которых проверяем наш подход, и почему мы выбрали именно их.

4.2. Исследовательские вопросы

По-английски называется *research questions*, в тексте можно ссылаться на них как RQ1, RQ2, и т. д. Необходимо сформулировать, чего мы хотели бы добиться работой (2 пункта будет хорошо):

RQ1 : правда ли предложенный в работе алгоритм лучше вот таких-то остальных?

RQ2 : насколько существенно каждая составляющая влияет на улучшения? (Если в подходе можно включать/выключать какие-то составляющие.)

RQ3 : насколько точны полученные приближения, если работа строит приближения каких-то штук?

и т.п.

Иногда в работах это называют гипотезами, которые потом проверяют. Далее в тексте можно ссылаться на исследовательские вопросы как RQ, это общепринятое сокращение.

4.3. Метрики

Как мы сравниваем, что результаты двух подходов лучше или хуже:

- Производительность.
- Строчки кода.
- Как часто алгоритм «угадывает» правильную классификацию входа.

Иногда метрики вырожденные (да/нет), это не очень хорошо, но если в области исследований так принято, то ладно. Если метрики хитрые (даже IoU или F_1-меру можно считать хитрыми), разберите их в обзоре, пояснив, почему выбраны именно такие метрики.

4.4. Результаты

Результаты понятно что такое. Тут всякие таблицы и графики, как в таблице 1. Обратите внимание, как цифры выровнены по правому краю, названия по центру, а разделители $\times$ и $\pm$ друг под другом.

Перед написанием данного раздела имеет смысл проконсультироваться с литературой по проведению экспериментов [6].

Скорее всего Ваши измерения будут удовлетворять нормальному распределению, в идеале это надо проверять с помощью критерия Колмогорова и т.п. Если критерий этого не подтверждает, то у Вас что-то сильно не так с измерениями, надо проверять кэши процессора, отключать Интернет во время измерений, подкручивать среду исполнения (англ. runtime), чтобы сборка мусора не вмешивалась и т.п. Если критерий удовлетворён, то необходимо либо указать мат. ожидание и доверительный/предсказывающий интервал, либо мат. ожидание и среднеквадратичное отклонение, либо, если совсем лень заморачиваться, написать, что все измерения проводились с погрешностью, например, в 5%. Не приводите слишком много значащих цифр (например, время работы в 239.1 секунды при среднеквадратичном отклонении в

Таблица 1: Производительность какого-то алгоритма при различных разрешениях картинок (меньше — лучше), в мс., CI=0.95. За пример таблички кидаем чепчики в честь Я. Кириленко

Resolution	TENG	LAPM	VOLL4
1920×1080	406.23 ± 0.94	134.06 ± 0.35	207.45 ± 0.42
1024×768	145.00 ± 0.47	39.68 ± 0.10	52.79 ± 0.10
464×848	70.57 ± 0.20	19.86 ± 0.01	32.75 ± 0.04
640×480	51.10 ± 0.20	14.70 ± 0.10	24.00 ± 0.04
160×120	2.40 ± 0.02	0.67 ± 0.01	0.92 ± 0.01

50 секунд выглядит глупо, даже если ваш любимый бенчмарк так посчитал).

Замечание: если у вас получится улучшение производительности в пределах погрешности, то это обязательно вызовет вопросы. Если погрешность получилась значительной (больше 10-15% от среднего), это тоже вызовет вопросы, на которые надо ответить, либо разобравшись, что не так (первый подозреваемый — мультимодальное распределение), либо более глубоко статистически проанализировав результаты (например, привести гистограммы).

В этом разделе надо также явно ответить на Research Questions или как-то их прокомментировать.

4.4.1. RQ1

Пояснения

4.4.2. RQ2

Пояснения

4.5. Обсуждение результатов

Чуть более неформальное обсуждение, то, что сделано. Например, почему метод работает лучше остальных? Или, что делать со случаями, когда метод классифицирует вход некорректно.

4.6. Угрозы нарушения корректности (опциональный)

Если основная заслуга метода, это то, что он дает лучшие цифры, то стоит сказать, где мы могли облажаться, когда

1. проводили численные замеры;
2. выбирали тестовый набор (см. *confirmation bias*).

Например, если мы делали замер юзабилити нашего продукта по методике System Usability Scale, но в эксперименте участвовали Ваши друзья, которые хотят Вас порадовать, честно напишите об этом. Честно напишите, какие меры были приняты для минимизации рисков валидности результатов эксперимента и, если это содержательно, почему их не удалось полностью исключить.

4.7. Воспроизводимость эксперимента

Это настолько важно, что заслуживает тут своего подраздела (в самой работе не надо, это должно естественно вытекать из разделов выше) — эксперимент должно быть можно повторить и получить примерно такие же результаты, как у Вас. Поэтому выложите свой код, которым мерили. Выложите данные, на которых мерили. Или напишите, где эти данные взять. Напишите, как конкретно запустить, в каком окружении, что надо дополнительно поставить и т.п. Чтобы любой второкурсник мог выполнить все пункты и получить тот же график/таблицу, что у Вас. Не обязательно это делать прямо в тексте, можете в README своего репозитория, но где-то надо.

Код модуля в составе дисциплины, практики и т.п.	Трудоёмкость	Контактная работа обучающихся с преподавателем										Самостоятельная работа					Объем активных и интерактивных
		лекции	семинары	консультации	практические занятия	лабораторные работы	контрольные работы	коллоквиумы	текущий контроль	промежуточная аттестация	итоговая аттестация	под руководством преподавателя	в присутствии преподавателя	с использованием методических	текущий контроль	промежуточная аттестация	
Семестр 3	2	30								2				18		20	10
		2–42								2–25				1–1		1–1	
Итого	2	30								2				18		20	10

Таблица 2: Если таблица очень большая, то можно её изобразить на отдельной портретной странице. Не забудьте подробное описание, чтобы содержимое таблицы можно было понять не читая весь текст.

Заключение

Обязательно. Список результатов, который будет либо один к одному соответствовать задачам из раздела 1, либо их уточнять (например, если было «выбрать», то тут «выбрано то-то»).

- Результат к задаче №1.
- Результат к задаче №2.
- и т.д.

Если работа на несколько семестров, отчитывайтесь только за текущий. Можно в свободной форме обрисовать планы продолжения работы, но не увлекайтесь — если работа будет продолжена, по ней будет ещё один отчёт.

В заключении *обязательна* ссылка на исходный код, если он выносится на защиту, либо явно напишите тут, что код закрыт. Если работа чисто теоретическая и это понятно из решённых задач, про код можно не писать. Обратите внимание, что ссылка на код должна быть именно в заключении, а не посреди раздела с реализацией, где её никто не найдёт.

Старайтесь оформить программные результаты работы так, чтобы это был один репозиторий или один пуллреквест, правильно оформленный — комиссии тяжело будет собирать Ваши коммиты по всей истории. А если над проектом работало несколько человек и всё успело изрядно перемешаться, неизбежны вопросы о Вашем вкладе.

Заключение люди реально читают (ещё до «основных» разделов работы, чтобы понять, что же получилось и стоит ли вообще работу читать), так что оно должно быть вылизано до блеска.

Список литературы

[1] Dreyer Derek. How To Write Papers So People Can Read Them. — 2020. — URL: https://www.youtube.com/watch?v=XpgJ31GKPWI (дата обращения: 21 ноября 2021 г.).

[2] Jhala Ranjit. How to Design Talks. — 2020. — URL: https://www.youtube.com/watch?v=aFT79TmffPk (дата обращения: 21 ноября 2021 г.).

[3] Jones Simon Peyton. How to Give a Great Research Talk. — URL: https://www.microsoft.com/en-us/research/academic-program/give-great-research-talk/ (дата обращения: 21 ноября 2021 г.).

[4] Jones Simon Peyton. How to Write a Great Research Paper. — URL: https://www.microsoft.com/en-us/research/academic-program/write-great-research-paper/ (дата обращения: 21 ноября 2021 г.).

[5] Кознов Д. В. Слайды. — URL: https://drive.google.com/drive/folders/1yyhnycFC-LpT6d6aIy9-lW7dM8lWkL7w?usp=sharing (дата обращения: 16 декабря 2023 г.).

[6] Смирнов К. К. Заметки о проведении экспериментов. — URL: https://github.com/spbu-se/measurements (дата обращения: 16 января 2024 г.).